

LAB2 실험 전 보고서

순차 회로와 상태 기계 사전 시뮬레이션

과목 전자회로설계실험 2

제출일 2026년 9월 21일

성명 _____

학번 _____

1 보고서 범위와 결론

이 보고서는 LAB2의 필수 실험 01 - 08을 대상으로 정상 회로의 기능, 입력 조건, 자기검사 결과와 교안에서 지정한 수정 실험을 정리한다. 여덟 정상 설계 모두 기록된 PASS와 종료 시각이 확인되었다. LAB2-02부터 LAB2-08까지는 교안이 지정한 한 곳을 바꿔 원본 테스트벤치에서 예상한 첫 FAIL을 확인했고, RTL 복구 뒤 정상 PASS를 다시 확인했다. LAB2-01 교안에는 별도 수정 실험이 없어 고장 주입을 하지 않았다.

PASS는 기록된 테스트벤치가 지정한 비교를 통과했다는 뜻이다. 보드 핀, 입력 전처리, FPGA 구현이나 모든 가능한 입력 조합까지 검증했다는 뜻은 아니다. 실험 파일은 2026년 9월 21일 로컬 작업 트리 스냅샷이므로, 제출 전 변경 파일을 commit하고 제출용 tag를 붙여 재현 근거를 고정해야 한다.

실험	검증 요약
01 4비트 업다운 카운터	정상 36회 · 종료 356 ns · 수정 없음
02 클록 분주기	정상 129회 · 종료 436 ns · 16 ns · minimum divisor duty 검사. 기대 div2=1, 실제 div2=0. 원본 TB가 첫 불일치에서 중단했다. RTL을 원복해 LAB2_PASS clock_divider checks=129를 다시 확인했다.
03 레지스터 쌍	정상 7회 · 종료 66 ns · 26 ns · transfer stored not live input 검사. 기대 stored=A,value=A, 즉 8'hAA였으나 실제 stored=A,value=3, 즉 8'hA3였다. RTL 원복 후 LAB2_PASS register_pair checks=7을 확인했다.
04 4비트 시프트 레지스터	정상 8회 · 종료 106 ns · 16 ns · input enters MSB 검사. 첫 입력 1 뒤 기대값은 1000, 실제 변경 RTL은 0001이다. RTL 원복 후 LAB2_PASS shift_register4 checks=8, 종료 106 ns를 다시 확인했다.
05 4비트 PISO 레지스터	정상 114회 · 종료 976 ns · 86 ns · MSB first before edge 검사. word=1에서 기대 serial_out=0, 수정 RTL의 실제 serial_out=1이다. 정상 RTL 복구 후 LAB2_PASS piso4 checks=114, 종료 976 ns를 확인했다.

실험	검증 요약
06 Moore 순환 상태기계	정상 23회 · 종료 226 ns · 46 ns · S1 to S2 검사. enable=1, advance=1 에지 뒤 기대값 10, 실제값 00이다. 원복 뒤 LAB2_PASS moore_cycle checks=23, 종료 226 ns를 확인했다.
07 Mealy 토글 상태기계	정상 11회 · 종료 67 ns · 7 ns · S0 input changes between clocks 검사. state=0, bit_in=1, enable=0에서 기대 {state,value}=010, 실제 001이다. RTL 복구 후 LAB2_PASS mealy_toggle checks=11, 종료 67 ns를 확인했다.
08 8자리 7세그먼트 스캔	정상 194회 · 종료 1306 ns · 6 ns · reset blanks digit zero 검사. 기대 select=00, 실제 select=01이며 index=0에서 첫 자리가 켜졌다. RTL 원복 후 LAB2_PASS segment_scan8 checks=194, 종료 1306 ns를 확인했다.

2 실험 환경과 검증 방법

코어 RTL은 Verilog로 작성하고 VS Code의 실행 작업에서 Icarus Verilog (iverilog/vvp)로 컴파일·시뮬레이션했다. 테스트벤치는 VCD를 저장하고 VaporView에서 clk, reset, 제어 입력, 상태 또는 출력 파형을 비교했다. 보드 top과 XDC는 프로젝트에 포함되어 있지만 이 코어 TB는 XDC를 읽지 않는다. 따라서 PASS는 핀 할당과 실제 보드 입출력의 검증을 포함하지 않는다.

TB의 clk 주기는 10 ns로 설정되어 있다. 순차 로직은 posedge clk에서 nonblocking assignment로 다음 상태를 예약하고, 검사 task는 에지 뒤 1 ns 기다린 후 출력을 비교한다. 이 시간 간격은 같은 에지에서 여러 레지스터를 갱신할 때 기존 상태와 새 상태를 구분해 관찰하도록 한다. 실험별 예외나 입력 순서는 각 장에 적었다.

3 소스 출처와 실행 스냅샷

항목	식별 정보
템플릿 원격	https://github.com/Glaysia/fpga-lab-template.git
기준 태그·커밋	v2.0.1 · 86c15c556e2edf1564a203180690f7e c4ad056af
상위 저장소 remote	https://github.com/tae2yi/ UOS_ece2.git
상위 저장소 HEAD	2609ab4b461c5add334791ebf8d69a 721434d9fc

템플릿 주소와 태그는 코드의 기준 출처다. 실험별 source, 테스트벤치와 evidence가 기준 태그 상태와 같다고 간주하지 않으며, 현재 보고된 결과는 각 실험 폴더의 normal/modified/recovered 로그와 VCD에 연결한다.

LAB2-01 4비트 업다운 카운터

설계 목표와 회로 구조

동기식 리셋과 enable 제어를 갖는 4비트 업다운 카운터를 설계한다. down이 0이면 증가하고 1이면 감소하며, 4비트 범위 끝에서 자연스럽게 순환한다.

상승 에지에서 rst를 가장 먼저 확인한다. rst가 1이면 value를 0으로 만들고, 그렇지 않을 때 enable이 1이면 down 방향에 따라 1을 더하거나 뺀다. enable이 0인 동안에는 값이 유지된다. 보드용 입력 전처리기와 top은 별도 RTL이며 이 TB는 counter4 코어만 검사한다.

핵심 코드

정상 코어 counter4의 상태 갱신 또는 출력 생성 부분이다. 보드 연결 모듈과 입력 전처리 코드는 생략했다.

```
always @(posedge clk) begin
    if (rst) value <= 4'd0;
    else if (enable) begin
        if (down) value <= value - 4'd1;
        else value <= value + 4'd1;
    end
end
```

입력 조건과 예상 결과

TB의 clk 주기는 10 ns이다. reset 후 enable을 켜고 down=0으로 16번 검사해 0, 1, ..., 15, 0을 확인한다. 이어 down=1로 16번 검사해 15, 14, ..., 0, 15가 되는지 확인한다. enable을 내리면 값이 유지되어야 하며, reset과 enable이 함께 1일 때 reset이 우선해야 한다.

정상 시뮬레이션 결과

LAB2_PASS counter4 checks=36. 종료 356 ns. 36개 자기검사에서 리셋, 양방향 증가·감소, 랩어라운드, hold 및 리셋 우선순위가 통과했다. input_frontend, XDC 핀 배치와 보드 동작을 검증한 결과는 아니다.

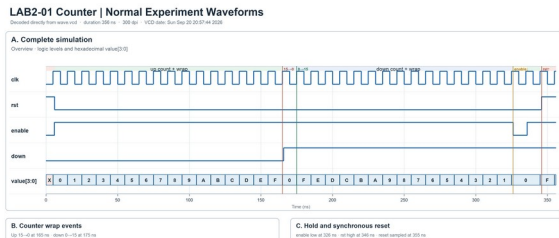


그림 1. 정상 전체 구간의 증가·감소와 wrap.

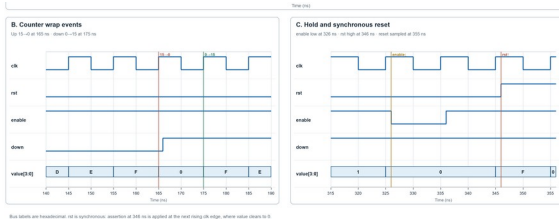


그림 2. wrap, hold 및 동기 reset 상세 구간.

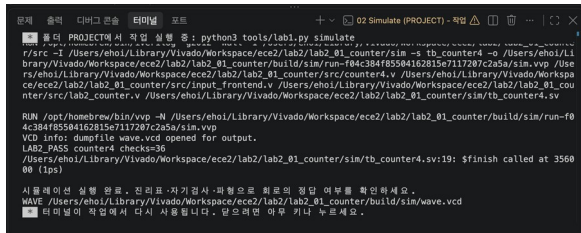


그림 3. LAB2-01 정상 실행의 PASS 및 종료 시각 터미널 로그.

수정 실험 및 예상 오류

교안에는 별도의 수정 실험이 제시되지 않는다. 정상 RTL과 TB를 그대로 두며, 임의의 고장을 주입하거나 수정 FAIL을 만들지 않았다.

수정 실행 결과

해당 없음

문제 해결 과정과 결론

정상 파형은 각 상승 에지에서만 value가 변하고, enable이 꺼진 동안 값을 보존함을 보인다. 4비트 산술에 따라 증가·감소 경계에서 순환한다. 검증 범위는 코어 TB에 한정된다.

LAB2-02 클록 분주기

설계 목표와 회로 구조

입력 클록을 정수 비율로 분주해 관찰용 사각파 divided와 한 클록 폭 enable 신호 tick을 만든다.

count는 DIVISOR 1까지 센 뒤 0으로 돌아간다. 짝수 DIVISOR에서 count가 절반 주기 경계에 도달하면 divided를 올리고 전체 주기 경계에서 내리므로 high·low 길이가 같다. tick은 count가 마지막 값일 때 1이며 다른 순차 회로의 enable로 사용한다. 테스트벤치에는 DIVISOR=10과 최소 허용값 2가 포함된다.

핵심 코드

정상 코어 clock_divider의 상태 갱신 또는 출력 생성 부분이다. 보드 연결 모듈과 입력 전처리 코드는 생략했다.

```
localparam integer WIDTH = $clog2(DIVISOR);
reg [WIDTH-1:0] count;
assign tick = !rst && (count == DIVISOR - 1);
always @(posedge clk) begin
    if (rst) begin count <= 0; divided <= 1'b0;
    end
    else begin
        if (count == DIVISOR - 1) count <= 0;
        else count <= count + 1'b1;
        if (count == DIVISOR/2 - 1) divided <=
            1'b1;
        else if (count == DIVISOR - 1) divided <=
            1'b0;
        end
    end
end
```

입력 조건과 예상 결과

TB clk는 10 ns 주기다. reset 뒤 DIVISOR=10에서 30 주기 동안 divided의 위상·50% duty 및 마지막 count에서의 tick을 확인한다. DIVISOR=2 인스턴스에서도 매 2주기 출력과 enable pulse를 확인하고, 중간 reset 후 처음부터 다시 시작하는지 검사한다.

정상 시뮬레이션 결과

LAB2_PASS clock_divider checks=129. 종료 436 ns. DIVISOR=10, 2의 duty와 위상, tick 시점·소비 pulse, reset 및 재시작 검사가 모두 통과했다.

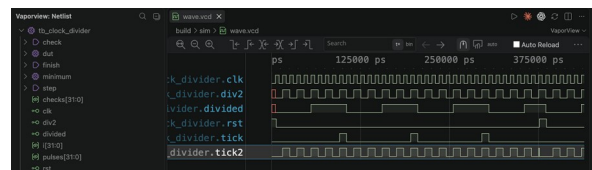


그림 4. 정상 VaporView 화면의 divided, tick 및 DIVISOR=2 출력·tick 파형.

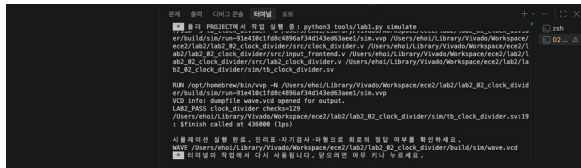


그림 5. LAB2-02 정상 실행의 PASS 및 종료 시각 터미널 로그.

수정 실험 및 예상 오류

교안 지시에 따라 divided 상승 비교식을 `count == DIVISOR/2 - 1`에서 `count == DIVISOR/2`로 잠시 바꾸고 TB는 유지한다. DIVISOR=2에서는 상승 조건과 하강 조건이 겹쳐 첫 활성 에지 이후에도 div2가 낮게 남는다.

수정 실험 결과

구분	내용
시점·검사	16 ns · minimum divisor duty 검사
기대	div2=1
실제	div2=0

16 ns · minimum divisor duty 검사. 기대 div2=1, 실제 div2=0. 원본 TB가 첫 불일치에서 중단했다. RTL을 원복해 LAB2_PASS clock_divider checks=129를 다시 확인했다.

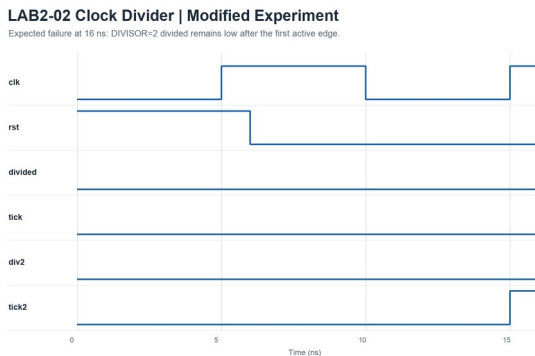


그림 6. 수정 파형에서 16 ns 실패 직전 div2는 계속 0이며 tick2가 15 ns에 올라간다.

문제 해결 과정과 결론

상태 카운트는 계속 주 클록의 상승 에지에서 갱신한다. 10분주 비교식은 상승 경계를 한 주기 늦춰 60 ns low, 40 ns high가 되며, DIVISOR=2의 검사는 경계 조건이 겹치는 결함을 잡아낸다.

LAB2-03 레지스터 쌍

설계 목표와 회로 구조

입력값 저장용 stored와 저장값 전달용 value를 분리해, 저장·전달 제어의 우선순위와 동시 동작을 확인한다.

두 4비트 레지스터는 모두 clk 상승 에지에서 갱신된다. reset은 둘 다 0으로 만든다. load와 transfer는 독립된 조건문이므로 동시에 동작할 수 있다. nonblocking 대입 때문에 같은 에지에서 value가 받는 것은 갱신 전 stored 값이다.

핵심 코드

정상 코어 register_pair의 상태 갱신 또는 출력 생성 부분이다. 보드 연결 모듈과 입력 전처리 코드는 생략했다.

```
always @(posedge clk) begin
    if (rst) begin
        stored <= 4'd0;
        value <= 4'd0;
    end else begin
        if (load) stored <= data_in;
        if (transfer) value <= stored;
    end
end
```

입력 조건과 예상 결과

clk 주기 10 ns. reset 뒤 data_in=A를 load해 stored=A,value=0을 확인한다. data_in을 3으로 바꾼 채 transfer하면 value=A여야 한다. load와 transfer를 같이 켜면 stored는 3으로 갱신되지만 value는 이전 A를 받아야 하며, 다음 전달 에지에는 value=3이 된다. 이후 hold와 reset 우선순위를 확인한다.

정상 시뮬레이션 결과

LAB2_PASS register_pair checks=7. 종료 66 ns. reset, load만 수행, 저장값 전달, 동시 동작에서 이전 stored 사용, 다음 에지 전달, hold, reset 우선순위를 확인했다.

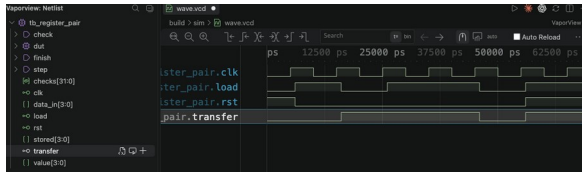


그림 7. 정상 VaporView 캡처. 신호 트리에는 stored와 value가 있으나 표시된 파형 행에는 포함되지 않았다.

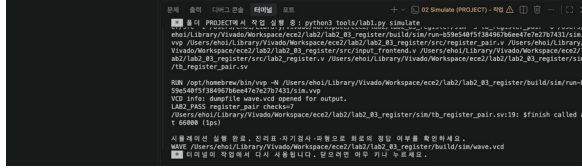


그림 8. LAB2-03 정상 실행의 PASS 및 종료 시각 터미널 로그.

수정 실험 및 예상 오류

교안 지시에 따라 transfer 대입의 우변을 stored에서 data_in으로 바꾸고 TB는 그대로 둔다. 이 수정은 등록된 값을 전달하지 않고 전달 에지의 입력을 직접 value에 저장한다.

수정 실행 결과

구분	내용
시점·검사	26 ns · transfer stored not live input 검사
기대	stored=A,value=A (8'hAA)
실제	stored=A,value=3 (8'hA3)

26 ns · transfer stored not live input 검사. 기대 stored=A,value=A, 즉 8'hAA였으나 실제 stored=A,value=3, 즉 8'hA3였다. RTL 원복 후 LAB2_PASS register_pair checks=7을 확인했다.

LAB2-03 Register Pair | Modified Experiment

Expected A at 25 ns, but live input 3 is routed to value; original TB fails at 26 ns.

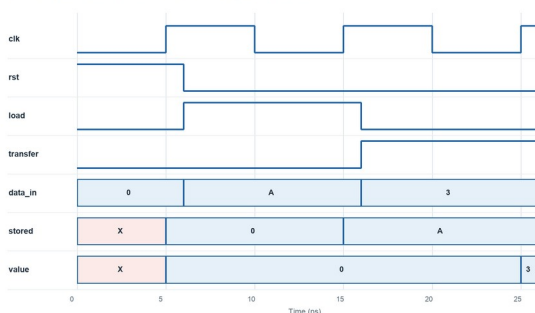


그림 9. 25 ns 전달 에지에서 stored=A와 data_in=3이 함께 보이고, 수정 회로의 value는 3으로 갱신된다.

문제 해결 과정과 결론

저장 단계와 전달 단계는 독립된 레지스터 동작이다. 동기 갱신 시 오른쪽 값은 에지 직전의 stored라는 점을 TB가 검사한다. 정상 스크린샷에서 출력 bus가 표시되지 않으므로 수정 파형과 보관 VCD를 함께 해석한다.

LAB2-04 4비트 시프트 레지스터

설계 목표와 회로 구조

직렬 입력 비트를 최상위 비트에 넣고, 매 enable 에지마다 기존 비트를 하위 방향으로 한 칸 이동한다.

동기 reset은 값을 0으로 만든다. enable이 1일 때만 {serial_in,value[3:1]}을 저장하므로 serial_in은 bit 3으로 들어가고 기존 bit 3:1은 bit 2:0으로 이동한다. enable=0이면 값은 유지된다.

핵심 코드

정상 코어 shift_register4의 상태 갱신 또는 출력 생성 부분이다. 보드 연결 모듈과 입력 전처리 코드는 생략했다.

```
always @(posedge clk) begin
    if (rst) value <= 4'd0;
    else if (enable)
        value <= {serial_in, value[3:1]};
end
```

입력 조건과 예상 결과

TB에서 reset 이후 enable=1로 serial_in=1을 인가하고 15 ns 상승 에지에서 샘플링한다. 입력 1은 value=1000이 되어야 한다. 이어 입력 비트를 이동하며 출력 상태를 검사하고, enable을 내린 구간에서는 값이 변하지 않아야 한다.

정상 시뮬레이션 결과

LAB2_PASS shift_register4 checks=8. 종료 106 ns. 입력이 MSB로 들어가고 이후 이동·유지 동작이 TB 기대 값을 통과했다.

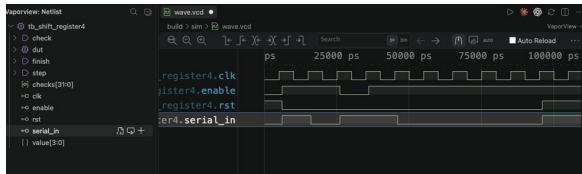


그림 10. 정상 실행 VaporView 파형과 PASS 로그.

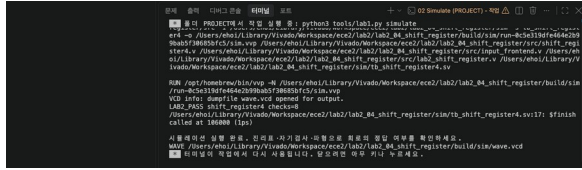


그림 11. LAB2-04 정상 실행의 PASS 및 종료 시각 터미널 로그.

수정 실험 및 예상 오류

교안의 수정은 다음 상태 연결을 {serial_in,value[3:1]}에서 {value[2:0],serial_in}으로 바꾸는 것이다. 즉 입력이 LSB로 들어가며 기존 비트는 반대 방향으로 이동한다. TB 기대값은 유지한다.

수정 실행 결과

구분	내용
시점·검사	16 ns · input enters MSB 검사
기대	value=1000
실제	value=0001

16 ns · input enters MSB 검사. 첫 입력 1 뒤 기대값은 1000, 실제 변경 RTL은 0001이다. RTL 원복 후 LAB2_PASS shift_register4 checks=8, 종료 106 ns를 다시 확인했다.

Shift Register: Reversed Shift Direction

First serial input is written to bit 0; unchanged TB expects bit 3.

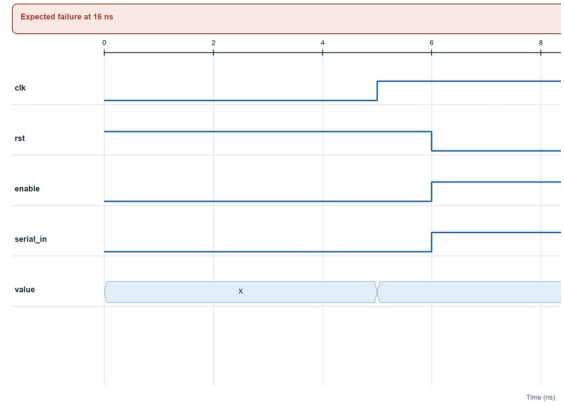


그림 12. 첫 enable 상승 에지에서 serial_in=1이 bit 0에 나타나며 16 ns 검사에서 실패한다. 전반 구간.

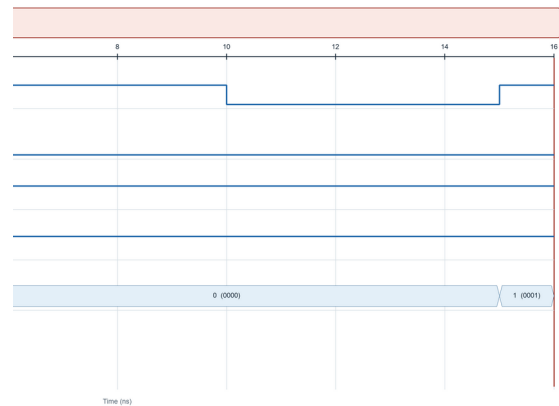


그림 13. 첫 enable 상승 에지에서 serial_in=1이 bit 0에 나타나며 16 ns 검사에서 실패한다. 실패 시점 확대 구간.

문제 해결 과정과 결론

연결 순서가 직렬 데이터의 유입 방향을 결정한다. 첫 입력만으로 어느 비트에 데이터가 들어갔는지 분명히 구분되며, 수정 실험 로그와 파형이 교안 예상과 일치한다.

LAB2-05 4비트 PISO 레지스터

설계 목표와 회로 구조

4비트 데이터를 병렬로 적재한 뒤 MSB부터 한 비트씩 직렬 출력한다.

serial_out은 value[3]에 연결된다. clk 상승 에지에서 reset, load, enable 순서로 동작을 선택한다. load가 우선하며 data_in을 병렬 적재한다. enable은 {value[2:0], 1'b0}로 왼쪽 이동하고 빈 LSB를 0으로 채운다.

핵심 코드

정상 코어 piso4의 상태 갱신 또는 출력 생성 부분이다. 보드 연결 모듈과 입력 전처리 코드는 생략했다.

```
assign serial_out = value[3];
always @(posedge clk) begin
    if (rst) value <= 4'd0;
    else if (load) value <= data_in;
    else if (enable)
        value <= {value[2:0], 1'b0};
end
```

입력 조건과 예상 결과

TB는 데이터를 load한 뒤 네 번 enable해 MSB 우선으로 직렬 비트를 읽는다. 먼저 word=0을 전달한 뒤 word=1을 병렬 적재한다. word=1의 첫 직렬 비트는 bit[3]=0이어야 한다. 총 114개 비교로 reset, load, shift, 출력 순서를 확인한다.

정상 시뮬레이션 결과

LAB2_PASS piso4 checks=114. 종료 976 ns. 적재 우선순위와 MSB-first 네 비트 전송을 포함한 검사가 모두 통과했다.

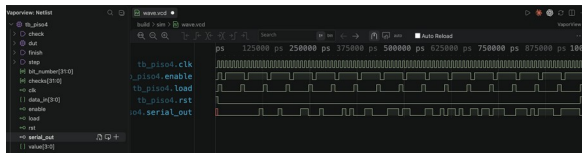


그림 14. 정상 실행에서 value와 MSB 직렬 출력의 동작을 확인한다.

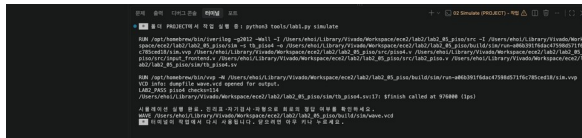


그림 15. LAB2-05 정상 실행의 PASS 및 종료 시각 터미널 로그.

수정 실험 및 예상 오류

교안의 수정은 serial_out을 value[3]에서 value[0]으로 바꾸는 것이다. TB는 MSB 우선 기대값을 유지하므로 LSB 출력은 첫 데이터에서 바로 어긋난다.

수정 실행 결과

구분	내용
시점·검사	86 ns · MSB first before edge 검사
기대	serial_out=0
실제	serial_out=1

86 ns · MSB first before edge 검사. word=1에서 기대 serial_out=0, 수정 RTL의 실제 serial_out=1이다. 정상 RTL 복구 후 LAB2_PASS piso4 checks=114, 종료 976 ns를 확인했다.

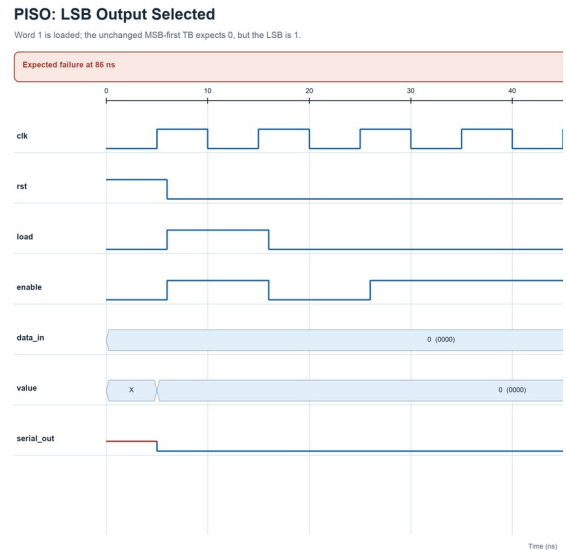


그림 16. word=1을 적재한 뒤 value=0001이며 LSB 출력이 1로 나타나 86 ns에 실패한다. 전반 구간.

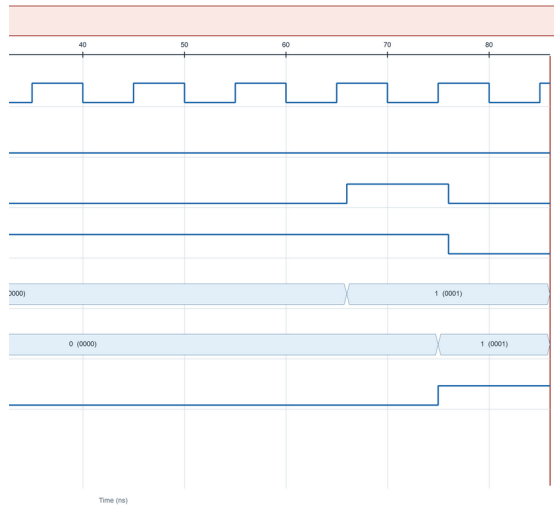


그림 17. word=1을 적재한 뒤 value=0001이며 LSB 출력이 1로 나타나 86 ns에 실패한다. 실패 시점 확대 구간.

문제 해결 과정과 결론

병렬 word의 첫 출력 위치를 지정하는 인덱스는 직렬 전송 순서를 결정한다. LSB와 MSB 차이는 word=0001 하나만으로 검출된다.

LAB2-06 Moore 순환 상태기계

설계 목표와 회로 구조

enable과 advance가 함께 활성화될 때 S0→S1→S2→S0 순환하는 2비트 Moore 상태기계를 구성한다.

상태 value는 동기 reset에서 00으로 초기화된다. 두 제어기가 모두 1인 상승 에지에서만 다음 상태로 간다. 나머지 입력에서는 현재 상태를 보존한다. 출력은 등록된 상태 자체이므로 입력 변화만으로 즉시 바뀌지 않는다.

핵심 코드

정상 코어 moore_cycle의 상태 갱신 또는 출력 생성 부분이다. 보드 연결 모듈과 입력 전처리 코드는 생략했다.

```
always @(posedge clk) begin
    if (rst) value <= 2'b00;
    else if (enable && advance) begin
        case (value)
            2'b00: value <= 2'b01;
            2'b01: value <= 2'b10;
            default: value <= 2'b00;
        endcase
    end
end
```

입력 조건과 예상 결과

TB는 reset으로 S0=00을 만든다. enable과 advance를 이용해 S0에서 S1, S1에서 S2, S2에서 S0로 진행하는 경우와 정지 조건을 검사한다. 출력 상태는 유효한 상승 에지 뒤에 바뀐다.

정상 시뮬레이션 결과

LAB2_PASS moore_cycle checks=23. 종료 226 ns. 상태 전이와 정지 조건에 대한 23개 검사가 통과했다.

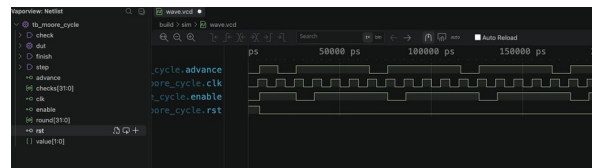


그림 18. 정상 상태 파형에서 상승 에지마다 상태가 순환한다.

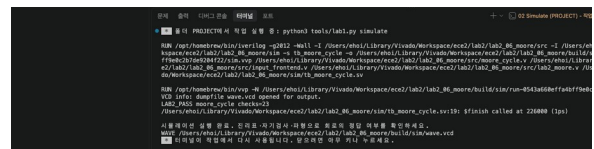


그림 19. LAB2-06 정상 실행의 PASS 및 종료 시각 터미널 로그.

수정 실험 및 예상 오류

교안 지시에 따라 S1=01의 다음 상태를 S2=10에서 S0=00으로 변경한다. TB의 S1→S2 기대값은 그대로 둔다.

수정 실행 결과

구분	내용
시점·검사	46 ns · S1 to S2 검사
기대	value=10
실제	value=00

46 ns · S1 to S2 검사. enable=1, advance=1 에지 뒤 기대값 10, 실제값 00이다. 원복 뒤 LAB2_PASS moore_cycle checks=23, 종료 226 ns를 확인했다.

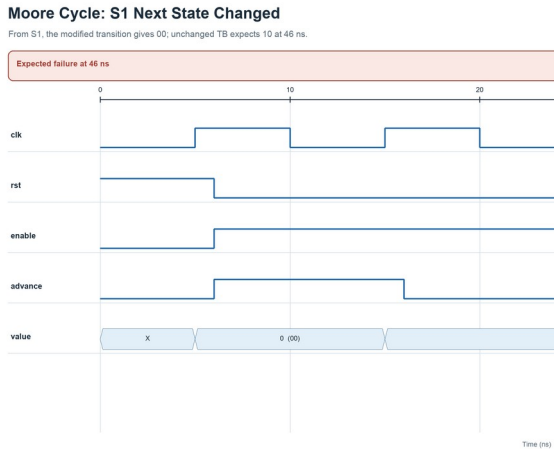


그림 20. S1에서 수정 전이값 00이 출력되어 46 ns 검사에서 S2 기대값과 다르다. 전반 구간.

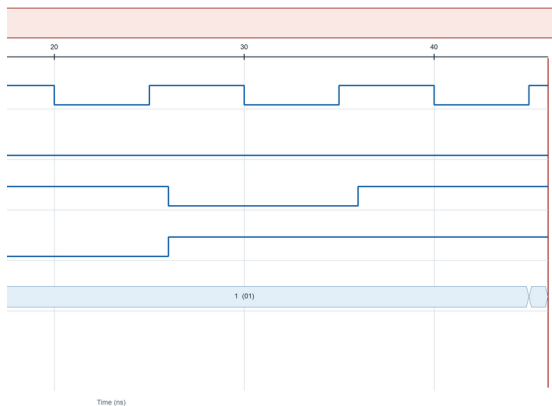


그림 21. S1에서 수정 전이값 00이 출력되어 46 ns 검사에서 S2 기대값과 다르다. 실패 시점 확대 구간.

문제 해결 과정과 결론

Moore 기계의 다음 상태는 현재 상태와 제어 입력에 따라 결정되며 출력 변화는 클럭 경계에서 발생한다. 한 전이만 잘못된 경우에도 해당 경로를 지정한 검사에서 검출된다.

LAB2-07 Mealy 토글 상태기계

설계 목표와 회로 구조

입력이 1일 때 enable에 따라 상태를 토글하고, 현재 상태와 입력을 함께 이용해 조합 출력을 생성한다.

state는 상승 에지에서만 갱신된다. reset은 S0을 선택하며 enable과 bit_in이 모두 1일 때 상태가 반전된다. value는 조합 논리다. bit_in=0이면 00, bit_in=1이면 현재 state에 따라 10 또는 01이 된다.

핵심 코드

정상 코어 mealy_toggle의 상태 갱신 또는 출력 생성 부분이다. 보드 연결 모듈과 입력 전처리 코드는 생략했다.

```
always @(posedge clk) begin
    if (rst) state <= 1'b0;
    else if (enable && bit_in) state <= ~state;
end
assign value = bit_in ?
    (state ? 2'b01 : 2'b10) : 2'b00;
```

입력 조건과 예상 결과

TB는 S0을 reset한 뒤 state가 그대로인 동안 bit_in을 0에서 1로 바꾼다. 이 경우 조합 출력은 클럭 사이에도 변해야 한다. 이후 enable과 bit_in이 함께 1인 상승 에지에서 상태가 토글되는지 확인한다.

정상 시뮬레이션 결과

LAB2_PASS mealy_toggle checks=11. 종료 67 ns. 상태 토글, 입력 의존 조합 출력, 클럭 사이 출력 변화를 포함한 11개 검사가 통과했다.

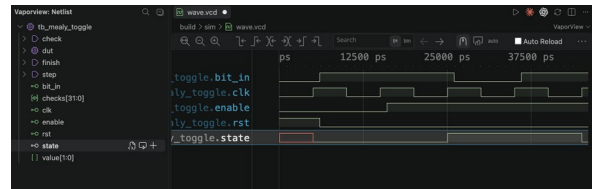


그림 22. 정상 파형에서 입력 변화가 value를 즉시 바꾸고 상태는 상승 에지에서 갱신된다.

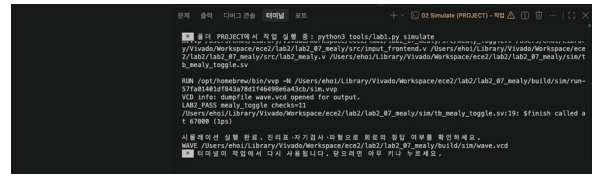


그림 23. LAB2-07 정상 실행의 PASS 및 종료 시각 터미널 로그.

수정 실험 및 예상 오류

교안은 bit_in=1일 때 S0과 S1의 출력을 서로 바꾸도록 한다. 원래 S0 출력은 10, S1 출력은 01이며 TB 기대값을 유지한다.

수정 실행 결과

구분	내용
시점·검사	7 ns · S0 input changes between clocks 검사
기대	{state,value}=010
실제	{state,value}=001

7 ns · S0 input changes between clocks 검사.
state=0, bit_in=1, enable=0에서 기대 {state,value}
=010, 실제 001이다. RTL 복구 후 LAB2_PASS
mealy_toggle checks=11, 종료 67 ns를 확인했다.

Mealy Toggle: Input-1 Outputs Swapped

At S0 with bit_in=1, the changed output is 01; unchanged TB expects 10.

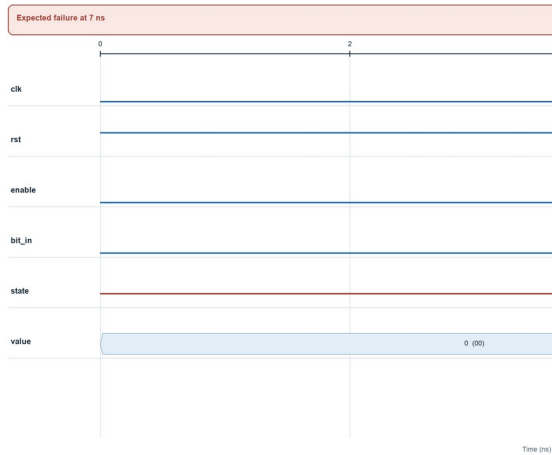


그림 24. S0에서 입력이 1로 바뀐 직후 수정 회로의 조합 출력 01
이 나타나 7 ns에 실패한다. 전반 구간.

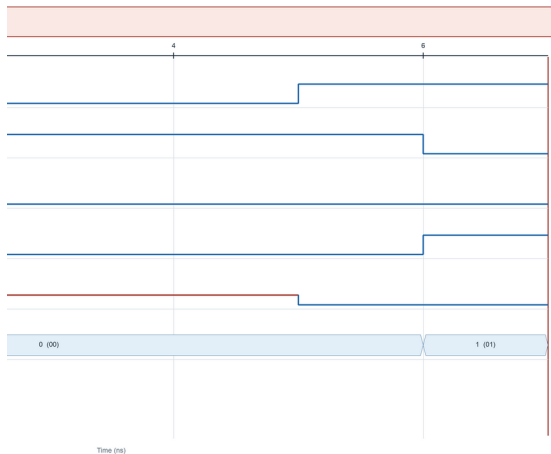


그림 25. S0에서 입력이 1로 바뀐 직후 수정 회로의 조합 출력 01
이 나타나 7 ns에 실패한다. 실패 시점 확대 구간.

문제 해결 과정과 결론

상태는 순차 논리, 출력은 조합 논리로 구현되어 있어 입력
변화와 상태 변화의 시점이 다르다. 클록 사이 검사는
Mealy 출력의 입력 의존성을 확인한다.

LAB2-08 8자리 7세그먼트 스캔

설계 목표와 회로 구조

8자리 표시기를 빠르게 순차 선택하고 각 4비트 숫자에 맞
는 세그먼트 패턴을 출력한다.

index가 현재 자리를 가리키고, digits에서 해당 4비트를
골라 16진수 디코더로 segments를 만든다. 한 자리 전
환 동안 blank를 활성화해 모든 select를 0으로 끄고, 다
음 구간에서 one-hot 자리를 켜다. 보드 polarity는
wrapper에서 적용한다.

핵심 코드

정상 코어 segment_scan8의 상태 갱신 또는 출력 생성
부분이다. 보드 연결 모듈과 입력 전처리 코드는 생략했
다.

```
if (rst) begin
    index <= 0;
    blank <= 1'b1;
end else if (enable) begin
    if (blank) blank <= 0;
    else begin
        blank <= 1;
        index <= index + 1'b1;
    end
end
nibble = digits >> (index * 4);
select = blank ? 8'h00 : (8'b0000_0001 << index);
```

입력 조건과 예상 결과

TB는 reset 동안 index=0, select=00으로 모든 자리가
꺼지는지 확인한다. 이후 enable로 blank와 one-hot
select가 번갈아 동작하고, 자리 index와 digits의 4비트
nibble에 맞는 segments 패턴이 나오는지 검사한다.

정상 시뮬레이션 결과

LAB2_PASS segment_scan8 checks=194. 종료
1306 ns. 리셋 blanking, 자리 선택, index 순환, 숫자
디코딩 검사가 통과했다.

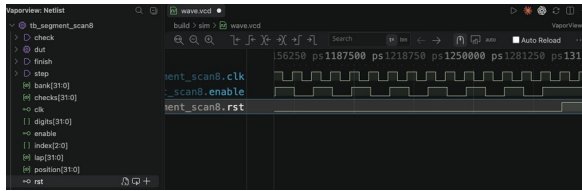


그림 26. 정상 VaporView 파형에서 blank 구간과 one-hot 자리 선택, 디코더 출력이 교대한다.

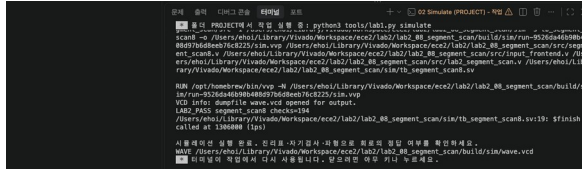


그림 27. LAB2-08 정상 실행의 PASS 및 종료 시각 터미널 로그.

수정 실험 및 예상 오류

교안에 따라 선택 신호의 blank 조건을 제거하고 one-hot 선택만 유지한다. TB는 변경하지 않아 reset 동안 모든 자리가 꺼져야 한다는 기대를 유지한다.

수정 실행 결과

구분	내용
시점·검사	6 ns · reset blanks digit zero 검사
기대	select=00
실제	select=01

6 ns · reset blanks digit zero 검사. 기대 select=00, 실제 select=01이며 index=0에서 첫 자리가 켜졌다. RTL 원복 후 LAB2_PASS segment_scan8 checks=194, 종료 1306 ns를 확인했다.

Segment Scan: Blanking Removed

During reset, the modified selector activates digit 0; the unchanged TB expects blank.

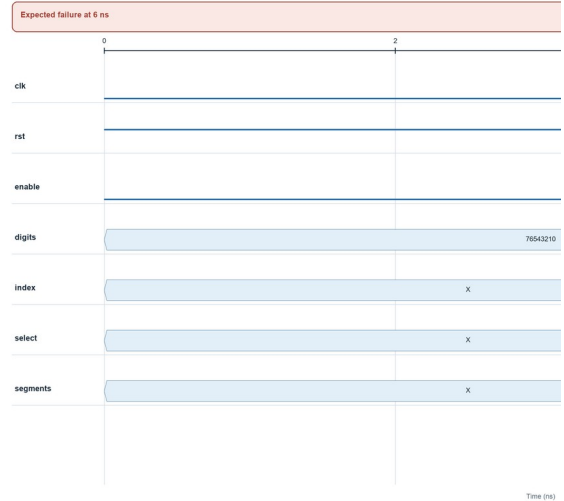


그림 28. 리셋 중 index=0, select=01 및 숫자 0 패턴 FC가 보이며 6 ns에 실패한다. 전반 구간.

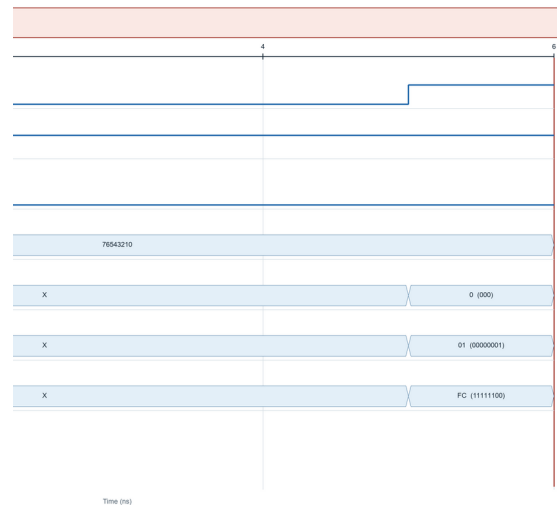


그림 29. 리셋 중 index=0, select=01 및 숫자 0 패턴 FC가 보이며 6 ns에 실패한다. 실패 시점 확대 구간.

문제 해결 과정과 결론

자리 전환 blanking은 표시값이 바뀌는 구간에 이전 자리 또는 영동한 세그먼트가 잠깐 켜지는 현상을 막는다. reset blanking 검사는 표시기 안전 상태를 첫 시점에 확인한다.

9 비교와 결론

여덟 실험은 상승 에지에서 상태가 바뀌는 순차 회로와 입력 변화에 따라 조합 출력이 바뀌는 Mealy 회로를 함께 다룬다. reset 우선순위, enable에 의한 유지, nonblocking assignment의 이전 상태 참조, 카운터 경계, 데이터 경로 방향, 상태 전이, 자리 blanking이 각각 독립된 검사로 확인되었다.

실험	핵심 관찰과 결과
01 카운터	방향·enable·wrap · 정상 36 / 356 ns · 수정 없음
02 분주기	주기·duty·tick · 정상 129 / 436 ns · 16 ns, div2 1→0
03 레지스터	저장값·이전 상태 · 정상 7 / 66 ns · 26 ns, AA→A3
04 시프트	직렬 입력 방향 · 정상 8 / 106 ns · 16 ns, 1000→0001
05 PISO	MSB-first · 정상 114 / 976 ns · 86 ns, 0→1
06 Moore	상태 순환 · 정상 23 / 226 ns · 46 ns, 10→00
07 Mealy	입력 의존 출력 · 정상 11 / 67 ns · 7 ns, 010→001
08 자리 스캔	blank·one-hot · 정상 194 / 1306 ns · 6 ns, 00→01

각 수정 실험에서는 한 동작만 바꾸고 테스트벤치를 유지했다. 따라서 첫 FAIL 시점은 변경된 동작과 기대값이 처음 갈라지는 지점이다. 정상 VCD와 수정 VCD를 함께 비교하고 RTL을 원복한 재실행까지 확인함으로써 의도된 오류와 복구 상태를 구분했다. 08A 통합 실습은 공통 프로젝트 흐름에만 언급되며 필수 실험 수에는 포함하지 않았다.

참고 문헌

M. Morris Mano and Michael D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog, 6th ed., Pearson, 2018, ISBN 978-0-13-454989-7.